

Tarea 2: Procesamiento y Fallback con Apache Kafka en Internet

Profesor: Nicolás Hidalgo

Ayudantes: Isidora González, Natalia Ortega, Joaquín Villegas, Vicente Díaz y Benjamín Aceituno

Introducción y Contexto

En la primera entrega se implementó una plataforma distribuida para el análisis de consultas geoespaciales utilizando el dataset Google Open Buildings. El sistema incorporó un mecanismo de caché para optimizar consultas repetidas sobre distintas zonas de la Región Metropolitana de Santiago, permitiendo reducir latencia y mejorar el rendimiento bajo distintos patrones de tráfico.

Sin embargo, el sistema desarrollado continúa dependiendo de una comunicación principalmente síncrona entre módulos. Esto implica que si uno de los servicios falla temporalmente, las consultas pueden retrasarse o incluso perderse antes de ser procesadas. Bajo escenarios de alta concurrencia, fallas temporales o sobrecarga del Generador de Respuestas, el sistema puede experimentar aumento de latencia, pérdida de consultas y degradación del throughput.

En esta segunda entrega se propone evolucionar la arquitectura incorporando Apache Kafka como sistema de colas de mensajes y desacoplamiento entre servicios. Mediante esta incorporación, se busca evitar la pérdida inmediata de consultas durante fallas temporales y mejorar la tolerancia a fallos del sistema bajo escenarios de alta carga.

El objetivo principal será implementar mecanismos de fallback y reintentos mediante colas Kafka, además de evaluar el comportamiento del sistema utilizando múltiples consumidores ejecutándose en paralelo.

1. Metodología

La segunda entrega estará enfocada en el procesamiento asíncrono de consultas y la tolerancia a fallos utilizando Apache Kafka.

El sistema deberá reutilizar la arquitectura y los servicios desarrollados en la Tarea 1, incluyendo:

- Generador de Tráfico
- Sistema Caché
- Generador de Respuestas
- Sistema de Métricas

No obstante, el flujo de consultas deberá modificarse para incorporar una capa de mensajería basada en Kafka. Se deberán implementar distintos escenarios de carga y fallos con el objetivo de analizar el comportamiento del sistema utilizando métricas experimentales.

La evaluación de esta etapa se centrará principalmente en:

1. Implementación de colas de reintento para consultas fallidas.
2. Recuperación automática ante fallos temporales.
3. Escalamiento horizontal utilizando múltiples consumidores Kafka.
4. Comparación experimental entre el sistema síncrono y el sistema basado en colas.
5. Evaluación del comportamiento del sistema bajo escenarios de alta carga y *spikes* de tráfico.

2. Descripción del Sistema

La arquitectura del sistema deberá mantenerse modular y desacoplada, incorporando Apache Kafka como intermediario entre el Generador de Tráfico y el procesamiento de consultas.

El sistema deberá considerar los siguientes componentes:

3. Descripción del Sistema

La arquitectura del sistema deberá mantenerse modular y desacoplada, incorporando Apache Kafka como intermediario entre el Generador de Tráfico y el procesamiento de consultas.

El sistema deberá considerar los siguientes componentes:

1. **Generador de Tráfico:** Genera consultas Q1–Q5 utilizando las distribuciones Zipf y uniforme implementadas en la Tarea 1.
2. **Kafka Producer:** Publica las consultas generadas en un tópico principal de Kafka.
3. **Sistema Caché:** Almacena respuestas previamente calculadas. Ante una solicitud, permite determinar si existe un *cache hit* o un *cache miss*. A diferencia de la Tarea 1, en esta etapa el sistema de caché no constituye el foco principal del análisis experimental. En cambio, se considerará como una configuración base sobre la cual se evaluará el comportamiento de la arquitectura asíncrona. Cada equipo deberá definir una configuración de caché adecuada, incluyendo tamaño de caché, TTL y Política de remoción, justificando debidamente la configuración seleccionada.
4. **Consumidores Kafka:** Consumen consultas desde Kafka e interactúan con el sistema caché. En caso de *cache hit*, retornan la respuesta inmediatamente. En caso de *cache miss*, derivan la consulta al Generador de Respuestas para su procesamiento.
5. **Generador de Respuestas:** Procesa las consultas delegadas por los consumidores cuando no existe información en caché, generando el resultado a partir de los datos precargados en memoria.
6. **Tópicos de Reintento:** Reciben consultas que fallan temporalmente debido a sobrecarga o indisponibilidad parcial del sistema. Estos tópicos permiten reenviar automáticamente las consultas para volver a procesarlas posteriormente.
7. **Dead Letter Queue (DLQ):** Almacena consultas que no lograron resolverse luego de alcanzar el número máximo de reintentos permitido.
8. **Sistema de Métricas:** Registra throughput, latencias, consultas recuperadas, reintentos, backlog de mensajes y eventos de fallo del sistema.

4. Flujo del Sistema

El Generador de Tráfico publicará consultas geoespaciales en un tópico principal de Kafka.

Los consumidores Kafka obtendrán las consultas desde dicho tópico y consultarán inicialmente el sistema caché:

- En caso de *cache hit*, la respuesta será retornada inmediatamente.
- En caso de *cache miss*, la consulta será enviada al Generador de Respuestas para su procesamiento.

Las fallas temporales podrán simularse, por ejemplo, deteniendo momentáneamente el Generador de Respuestas, agregando latencia artificial o provocando errores aleatorios durante el procesamiento.

Si ocurre una falla temporal durante el procesamiento:

- La consulta deberá reenviarse automáticamente a un tópico de reintento.
- Cada consulta tendrá un número máximo de intentos permitido.
- Las consultas que excedan dicho límite deberán enviarse a una Dead Letter Queue.

El objetivo de este mecanismo es evitar la pérdida inmediata de consultas frente a fallas temporales del sistema. Adicionalmente, el sistema deberá permitir ejecutar múltiples consumidores Kafka pertenecientes al mismo grupo de consumo, permitiendo distribuir automáticamente la carga de procesamiento.

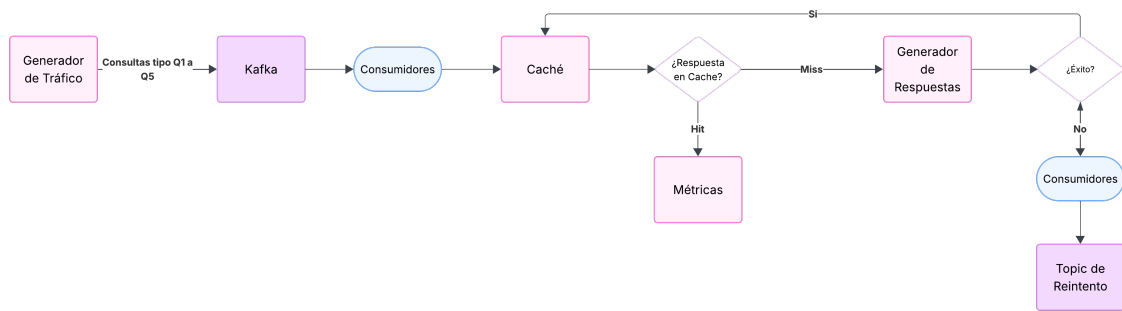


Figura 1: Flujo del sistema con la implementación de Apache Kafka como sistema de colas

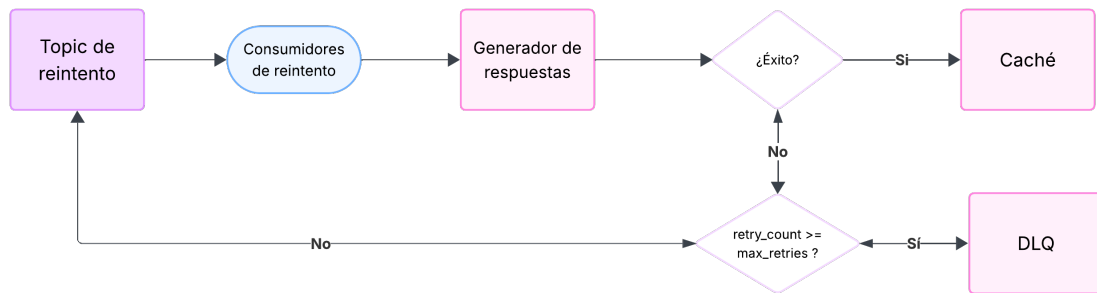


Figura 2: Flujo del sistema para la adición de consultas en la DLQ

5. Requerimientos Funcionales

5.1. Sistema de Reintentos

El sistema deberá implementar una estrategia de reintentos utilizando tópicos Kafka.

Cada vez que una consulta falle temporalmente, esta deberá reenviarse automáticamente para intentar procesarla nuevamente.

Cada consulta deberá incluir:

- Identificador único
- Número de reintentos realizados
- Timestamp de creación

No existe una única implementación correcta. La estrategia de reintentos y la organización de tópicos podrá definirse libremente, siempre que las decisiones estén correctamente justificadas.

El sistema deberá definir:

- Política de reintentos
- Número máximo de intentos
- Condiciones de envío a la DLQ

5.2. Procesamiento Paralelo

El sistema deberá soportar múltiples consumidores Kafka ejecutándose concurrentemente.

Todos los consumidores deberán pertenecer al mismo grupo de consumo, permitiendo balancear automáticamente la carga de trabajo.

Se deberá evaluar experimentalmente el impacto de aumentar la cantidad de consumidores sobre el rendimiento general del sistema.

5.3. Manejo de Backlog

El sistema deberá monitorear y analizar el backlog generado en Kafka bajo distintos escenarios de carga. Se deberá evaluar:

- Crecimiento del backlog ante sobrecarga del sistema
- Tiempo necesario para vaciar las colas tras una falla
- Impacto del backlog sobre latencia y throughput

6. Escenarios de Evaluación

El sistema deberá evaluarse bajo los siguientes escenarios:

1. **Sistema Base:** Arquitectura original síncrona sin Kafka.
2. **Kafka + 1 Consumer:** Procesamiento asíncrono utilizando un único consumidor.
3. **Kafka + Múltiples Consumers:** Comparación del throughput, latencia y backlog utilizando distintas cantidades de consumidores Kafka.
4. **Falla Temporal:** Simulación de caída parcial o total del Generador de Respuestas mientras continúan llegando consultas al sistema.
5. **Reintentos:** Consultas que fallan temporalmente antes de resolverse exitosamente.
6. **Spike de Tráfico:** Incremento repentino de la tasa de generación de consultas durante un período corto de tiempo.
7. **Recuperación ante Fallos:** Comparación entre la arquitectura síncrona de la Tarea 1 y la arquitectura basada en Kafka respecto a pérdida de consultas y capacidad de recuperación tras una falla temporal.

Análisis y Discusión

El análisis experimental deberá estar respaldado por métricas obtenidas durante la ejecución del sistema.

El análisis debe explicar no solo las métricas obtenidas, sino también las razones detrás del comportamiento observado en cada escenario.

El informe deberá responder, al menos, las siguientes preguntas:

- ¿Cómo cambia el comportamiento del sistema al incorporar Kafka respecto de la arquitectura síncrona original?
- ¿Qué impacto tienen los reintentos sobre la pérdida de consultas y la latencia total?
- ¿Cómo afecta el número de consumidores al throughput del sistema?
- ¿Qué ocurre con el backlog durante escenarios de alta carga o fallos temporales?
- ¿Qué ventajas y desventajas presenta una arquitectura basada en colas frente a la solución implementada en la Tarea 1?

Se deben incluir gráficos o visualizaciones comparativas entre escenarios para facilitar el análisis de resultados. Todo el análisis debe estar basado en las siguientes métricas:

Métrica	Definición
Throughput	Consultas procesadas exitosamente por segundo
Latencia p50/p95	Percentiles de tiempo de respuesta
Retry rate	Consultas reenviadas a tópicos de reintento
Recovery rate	Consultas recuperadas exitosamente tras fallos
DLQ rate	Consultas enviadas a la Dead Letter Queue
Backlog size	Cantidad de mensajes pendientes en Kafka
Recovery time	Tiempo necesario para vaciar la cola tras una falla

Requisitos de la entrega

La evaluación de este hito se basará en los siguientes entregables:

- **Informe Técnico:** Documento en formato PDF que describa la arquitectura implementada, las decisiones de diseño y el análisis experimental de resultados.
- **Video de Demostración:** Video mostrando el funcionamiento del sistema y los escenarios de falla y recuperación implementados.
- **Código Fuente:** Repositorio público en GitHub o GitLab con el código completo del proyecto.
- **Archivos de Despliegue:** Archivo docker-compose.yml y/o Dockerfile necesarios para ejecutar el sistema.

Reglas y Consideraciones

- **Fecha de Entrega:** La entrega se realiza el día Miércoles 3 de junio vía Canvas del curso.
- **Integrantes:** La tarea debe ser realizada en grupos de hasta 2 alumnos/as.
- **Tecnología:** Se requiere Docker y Apache Kafka. El stack interno queda a elección del equipo, siempre que cada decisión esté correctamente justificada.
- **Ética y Autoría:** Se debe respetar el reglamento de la universidad respecto a plagio y autoría.